

ARIA Project Report

Generated on: 7 August 2026

Project: ARIA - AI-powered finance assistant, tutor, market analyst, and equity researcher

Repository reviewed: D:\IIIT SRI CITY\BTP_clean\ARIA_CLEAN

Verification status: pytest completed successfully with 50 passed tests.

1. Executive Summary

ARIA is currently a working Streamlit-based prototype that combines four major functional modules behind a single chat interface:

1. Personal Finance Assistant
2. Intelligent Financial Tutor
3. Market Analysis Assistant
4. Equity Research Assistant

The project also includes voice input, text-to-speech output, and a browser-based 3D avatar for conversational mode.

The backend is much stronger than the current avatar layer. The finance, tutor, market, and equity research modules have real routing, structured pipelines, trained local model artifacts, real market/fundamental data paths, and a passing test suite. The avatar is currently a low-cost/free visual layer built around a local GLB model and browser-side TalkingHead/HeadAudio libraries. That explains why it feels anime/cartoon-like and why lip sync is weak or unreliable: the avatar asset is not a professional human presenter model, the lip sync depends on browser audio permissions and an experimental audio-driven viseme detector, and there is no production-grade facial animation stack.

2. What Has Been Built

2.1 Streamlit Application Interface

Files:

- interface/streamlit_app.py
- interface/components.py
- main.py

What is done:

- A Streamlit app is built as the main user interface.
- The app has three tabs:
 - Chat
 - Your Profile
 - Your Progress
 - Users can ask questions from a single chat box.
 - The app supports normal mode and conversational mode.
 - Chat history is stored per browser session using a generated session id.
 - Example questions are provided for all major modules.
 - Responses are rendered into structured cards when the LLM returns known sections such as Insight, Analysis, Recommendation, and Risk.
 - Tutor answers are rendered with TAXAL-style sections: Cognitive, Functional, and Causal.
 - Audio input is supported through Streamlit's audio_input.
 - Audio output is generated through TTS and played back in the UI.

- In conversational mode, the generated TTS audio is passed to the avatar iframe.

How it is done:

- interface/streamlit_app.py loads the orchestrator dynamically so pipeline code changes are picked up on Streamlit reruns.
- _handle_query() delegates every query to core.orchestrator.handle_query().
- _speech_text() cleans Markdown-like LLM output into a more natural TTS script.
- render_response() parses known response section labels and formats them as visual cards.
- render_profile_panel() saves finance profile inputs.
- render_progress_panel() shows per-concept tutor mastery.

Scale currently supported:

- Single-user or small-demo Streamlit usage.
- Each browser tab gets an independent session id.
- No multi-user authentication or production account system yet.
- State is lightweight and local, not enterprise-grade.

What needs to be built:

- Proper user login and persistent user accounts.
- Production deployment configuration.
- Stronger UI around source transparency, error states, and data freshness.
- Better separation between demo session state and long-term user data.
- Better mobile layout and accessibility verification.

3. Core Query Routing and Orchestration

Files:

- core/router.py
- core/orchestrator.py
- shared/intent_classifier.py
- shared/company_resolver.py

What is done:

- A central router chooses which module should answer a query.
- Personal finance and tutor routing use the shared intent classifier.
- Broad market routing detects index, sector, and macro-market questions.
- Equity research routing detects company-specific stock and fundamental questions.
- The orchestrator dispatches to one of four pipelines:
 - modules.finance.pipeline
 - modules.tutor.pipeline
 - modules.market.pipeline
 - modules.equity_research.pipeline
 - Conversational mode rewrites follow-up queries using recent chat history so questions like "what about debt?" can become self-contained.

How it is done:

- route_query() calls classify_intent().
- is_fundamental_query() detects balance-sheet and financial statement language.
- is_investment_query() detects live stock price and market-cap style language.
- is_broad_market_query() detects Nifty, Sensex, sector, election, RBI, inflation, GDP, and similar macro terms.
- resolve_company() is used before final dispatch so company-specific questions can be sent to equity research.

Scale currently supported:

- Rule-based routing works well for common demo queries.
- 50 automated tests include router and adversarial cases.
- The routing is not yet a trained production NLU classifier.

What needs to be built:

- A stronger intent evaluation dataset.
- Confusion-matrix reporting across finance/tutor/market/equity/general queries.
- Confidence scores and clarification prompts when routing is uncertain.
- More company alias coverage.

4. Module 1 - Personal Finance

Files:

- modules/finance/pipeline.py
- modules/finance/orchestrator.py
- modules/finance/risk_model.py
- modules/finance/forecast_model.py
- modules/finance/anomaly.py
- modules/finance/budget_optimize.py
- modules/finance/sip_emi_calc.py
- modules/finance/tax.py
- modules/finance/train/train_xgb_risk.py
- modules/finance/train/train_lstm_spend.py
- scripts/generate_synthetic_transactions.py

What is done:

- Risk profiling using XGBoost.
- Spending forecast using a small LSTM.
- Anomaly detection using Isolation Forest.
- Budget allocation using linear programming.
- SIP planning using deterministic financial math.
- EMI and SIP calculators.
- Indian tax calculation support.
- Free-text extraction of income, EMI, age, and related finance values.
- Saved financial profile fields are reused across turns.
- Known finance concept answers for terms such as SIP and EMI.
- LLM narration converts computed results into user-facing Insight, Analysis, Recommendation, and Risk sections.

How it is done:

- modules.finance.pipeline.run_pipeline() first tries to build a structured UserFinancialInput.
- If enough profile data exists, it calls modules.finance.orchestrator.run().
- The orchestrator combines:
 - risk_model.predict()
 - forecast_model.predict()
 - anomaly.detect_for_user()
 - budget_optimize.optimize_budget()
 - tax.compute()
 - SIP plan construction
 - The final result is narrated through the shared LLM client.
 - If structured finance data is missing, the pipeline falls back to SIP heuristics, known-

concept explanations, or a generic finance prompt.

Scale currently supported:

- Local trained artifacts exist under models/finance:
- xgb_risk_profile.json
- xgb_risk_profile.artifacts.joblib
- lstm_spend_forecast.pt
 - Training is designed to run on CPU.
 - Data is synthetic, not real bank transaction data.
 - Good for prototype and BTP demonstration scale.
 - Not yet validated on real user financial histories.

What needs to be built:

- Real or institution-grade anonymized transaction dataset.
- Proper consent-based data ingestion from CSV, bank statements, UPI exports, or account aggregator APIs.
- Model performance metrics in the app/report: accuracy, precision, recall, MAE/RMSE, anomaly precision.
- Financial planning assumptions screen.
- Explainability panel for risk model features.
- Production guardrails against personalized investment advice.
- More robust handling of inflation, goals, liabilities, insurance, emergency funds, and asset allocation.

5. Module 2 - Intelligent Financial Tutor

Files:

- modules/tutor/pipeline.py
- modules/tutor/orchestrator.py
- modules/tutor/retriever.py
- modules/tutor/knowledge.py
- modules/tutor/teaching.py
- modules/tutor/explainer.py
- modules/tutor/quiz.py
- modules/tutor/train/train_lstm_kt.py
- modules/tutor/train/train_dqn_teacher.py
- scripts/build_concept_kb.py
- scripts/generate_kt_sequences.py
- data/raw/concepts_kb/concepts.json

What is done:

- A financial concept knowledge base exists with 50 concepts.
- Concept retrieval uses Sentence-BERT and FAISS.
- Student mastery is estimated from quiz/history interactions.
- Teaching action selection uses a DQN policy.
- Explanations are generated in TAXAL format:
- Cognitive
- Functional
- Causal
 - Quiz generation is available.
 - User learning progress is stored and shown in the Streamlit progress tab.

How it is done:

- `modules.tutor.pipeline.run_pipeline()` calls `modules.tutor.orchestrator.handle()`.
- The orchestrator retrieves the best matching concept.
- It loads the user's learning state.
- It computes mastery with `mastery_vector()`.
- It chooses a teaching action with `teaching.choose_action()`.
- It generates an explanation with `explainer.explain()`.
- If the chosen action is quiz-based, it generates a quiz item.

Scale currently supported:

- 50 finance concepts are present in the concept KB.
- Local trained tutor artifacts exist under `models/tutor`:
- `lstm_kt.pt`
- `dqn_teacher.zip`
 - Suitable for controlled concept-teaching demos.
 - Not yet a full-scale adaptive learning platform.

What needs to be built:

- Larger concept graph with more prerequisites and difficulty calibration.
- Real student interaction dataset.
- A quiz answer UI flow that records answers directly from the chat.
- Learning analytics: mastery over time, weak areas, revision scheduling.
- Evaluation against expert-written explanations.
- More depth levels for advanced finance learners.

6. Module 3 - Market Analysis

Files:

- `modules/market/pipeline.py`
- `modules/market/analyzer.py`

What is done:

- Broad market questions are handled separately from company-specific stock research.
- Real Nifty and Sensex data is fetched through `yfinance`.
- Sector basket snapshots are supported for IT, banking, auto, pharma, and FMCG.
- Economic Times public market RSS headlines are used for broad market news.
- LLM narration summarizes the current market context.

How it is done:

- `MarketAnalyzer.analyze()` detects index and sector terms in the query.
- For index queries, it fetches 5-day OHLC history and computes trend/change.
- For sector queries, it fetches representative large-cap tickers and computes equal-weight 5-day average change.
- For general market queries, it fetches both Nifty and Sensex snapshots.
- Market headlines are pulled from the Economic Times RSS feed.

Scale currently supported:

- Works for common Indian market demo queries.
- Data freshness depends on `yfinance` and RSS availability.
- Sector analysis is a representative basket, not an official sector index.

What needs to be built:

- TTL caching instead of simple in-memory `lru_cache`.
- Official or more reliable market data source for production.

- More sectors and official sector index mapping.
- Technical indicators such as SMA, RSI, MACD, volatility, breadth, and volume analysis.
- Data timestamp display in the UI.
- Clear distinction between delayed data and live data.

7. Module 4 - Equity Research

Files:

- modules/equity_research/pipeline.py
- modules/equity_research/financial_pipeline.py
- modules/equity_research/screener_adapter.py
- modules/equity_research/data_normalizer.py
- modules/equity_research/metric_router.py
- modules/equity_research/calculations.py
- modules/equity_research/investment.py

What is done:

- Company-specific questions are routed to equity research.
- Fundamental questions use Screener.in data.
- Live stock price/news questions use yfinance.
- Company search supports ticker detection and Yahoo Finance search.
- An LLM-assisted fallback guesses likely listed Indian company names from noisy user queries.
- Fundamental data is normalized by fiscal year.
- Metric-specific calculations are routed based on query text.
- Output is grounded in retrieved numbers and constrained to structured sections.

How it is done:

- equity_research.pipeline.run_pipeline() resolves a company/ticker.
- Fundamental queries call process_financial_query().
- Live price/news queries call investment_module().
- Screener data is normalized, fiscal years are extracted, a metric route is chosen, and a grounded prompt is generated.
- yfinance provides current price, PE ratio, market cap, recent history, and news headlines.
- Sentiment is classified by the LLM from headlines.

Scale currently supported:

- Strong for demonstration on Indian listed companies when data sources respond.
- Reliability depends on Screener.in scraping and yfinance behavior.
- Uses in-memory caching without production-grade freshness guarantees.

What needs to be built:

- Robust data provider integration with service-level reliability.
- Company master database with NSE/BSE symbols and aliases.
- More fundamental ratios and multi-year trend analysis.
- Source links and data timestamps in every answer.
- Better fallback when Screener or Yahoo changes its structure.
- Investment compliance layer to avoid buy/sell recommendations.

8. LLM Layer

Files:

- ai/llm/groq_client.py

- ai/llm/prompt_templates.py
- ai/llm/audio_script.py

What is done:

- Groq is used as the primary LLM provider.
- Gemini is used as fallback.
- Prompt templates exist for finance, tutor, market, and equity research.
- Audio script generation exists to make spoken output cleaner than raw Markdown.

How it is done:

- Pipelines build structured context.
- Context is inserted into prompt templates.
- generate_response() handles LLM provider calls and fallback behavior.
- The UI handles provider errors with a clear error banner.

Scale currently supported:

- Good for prototype and demo use.
- Depends on API keys and free-tier limits.
- Tests mock LLM calls; test suite does not call real providers.

What needs to be built:

- Provider observability: latency, failures, token usage.
- Prompt regression tests.
- Safer financial-disclaimer and compliance templates.
- Model output validation before rendering.
- Offline fallback responses for key deterministic calculations.

9. Speech Layer

Files:

- ai/speech/stt.py
- ai/speech/tts.py

What is done:

- Speech-to-text uses Whisper.
- Whisper loading is lazy and thread-safe.
- CPU and CUDA device selection are supported.
- ffmpeg path handling is included for Windows.
- Text-to-speech uses Edge-TTS as primary.
- pyttsx3 is available as offline fallback.
- TTS writes temporary MP3/WAV files for playback.

How it is done:

- Streamlit collects audio with st.audio_input.
- transcribe() converts audio to text.
- The resulting text is inserted into the query box.
- After a response is generated, generate_audio_script() prepares a speech version.
- synthesize() creates an audio file.
- The Streamlit app plays the audio.

Scale currently supported:

- Suitable for local demo usage.
- Whisper model defaults to base.en, which is manageable on CPU but not instantaneous.
- Temporary audio files are not managed like production media assets.

What needs to be built:

- Better microphone UX with recording status and transcription confidence.
- Audio cleanup and storage lifecycle.
- Streaming TTS instead of waiting for full synthesis.
- Voice selection UI.
- Better handling of multilingual Indian English/Hindi finance queries.

10. Avatar Layer - Current Problems and Root Cause

Files:

- interface/avatar/avatar.html
- interface/avatar/render.py
- interface/avatar/model.glb
- interface/avatar/NOTICE.md

Current state:

- The avatar is a browser-embedded 3D head/body model.
- It uses TalkingHead loaded from jsdelivr.
- It uses HeadAudio for audio-signal-driven lip sync.
- The local model.glb is injected as a base64 data URL.
- TTS audio is also injected as base64 into the iframe.
- Streamlit also renders a normal audio player as fallback.

Why it looks anime/cartoon-like:

- The visual appearance comes from the selected model.glb.
- The current model is not a professional presenter avatar, not a realistic scanned human, and not a polished corporate character.
- TalkingHead can animate a GLB, but it cannot automatically turn a cartoon/anime asset into a professional human avatar.
- The current app chose a free/local asset to avoid paid avatar APIs and quotas.

Why lip sync is not working properly:

- Browser autoplay rules block a new iframe AudioContext until the user clicks inside that iframe.
- The code explicitly shows a "Click to hear ARIA" button because Streamlit's parent-page click does not count as a gesture inside the iframe.
- HeadAudio lip sync is experimental and audio-signal based.
- It does not use phoneme timestamps from the TTS provider.
- The code relies on TalkingHead internal morph-target mappings such as head.mtAvatar.
- If the GLB model does not have correct mouth blendshapes/morph targets, lip sync will be weak or invisible.
- If CDN loading fails, WebGL fails, AudioWorklet fails, or the browser blocks audio, the avatar becomes mostly a moving 3D model without believable speech impact.

Why it feels low impact:

- No professional avatar identity.
- No realistic face material, lighting, camera composition, or clothing design.
- No production-grade animation direction.
- No emotional expression mapping from response intent.
- No gaze/gesture system connected to the content.
- No high-quality viseme track.
- The avatar is a bolt-on iframe, not a deeply integrated conversational experience.

What needs to be built for a professional avatar:

- Replace model.glb with a professional realistic GLB/VRM avatar designed for speech.
- Ensure the model has correct ARKit-style or TalkingHead-compatible facial blendshapes.
- Use a TTS provider or pipeline that returns word/phoneme timing.
- Drive mouth movement from phoneme/viseme timings instead of raw audio energy only.
- Add expression states: neutral, confident, concerned, explaining, warning.
- Add gaze, blink, head pose, and idle behavior that feels restrained and professional.
- Add a polished scene: realistic lighting, camera framing, background, and responsive sizing.
- Remove the repeated click problem by restructuring audio playback flow where possible, while still respecting browser autoplay rules.
- Add automated avatar diagnostics: model loaded, audio decoded, AudioContext running, lip-sync worklet loaded, morph targets detected.
- Add a fallback professional non-3D mode: high-quality voice with transcript and waveform if WebGL fails.

Recommended professional avatar path:

1. Short-term fix: replace the current GLB with a professional human-style model that has proper mouth morph targets.
2. Short-term fix: show clear status badges for "audio ready", "lip sync ready", and "model loaded" during development.
3. Medium-term fix: generate or obtain viseme timestamps and drive morph targets deterministically.
4. Medium-term fix: improve camera, lighting, clothes, background, and facial expression states.
5. Long-term fix: move to a production avatar stack such as a realistic WebGL avatar pipeline, HeyGen-style API, or a custom Ready Player Me/VRM + Rhubarb/viseme timing pipeline.

11. Testing and Verification

Current verification:

- Command run: pytest
- Result: 50 passed
- Runtime: 29.88 seconds

Coverage areas visible from tests:

- Finance anomaly detection
- Budget optimization
- Forecast model behavior
- Risk model behavior
- SIP and EMI calculations
- Tax calculations
- Router behavior
- Adversarial routing/finance cases
- Tutor explanation
- Tutor knowledge/mastery
- Tutor quiz generation
- Tutor retrieval

What still needs to be tested:

- End-to-end Streamlit UI behavior.
- Real LLM provider behavior under rate limits.

- Real yfinance/Screener/RSS failures.
- Voice input under noisy audio.
- TTS fallback behavior.
- Avatar loading and lip sync in actual browsers.
- Multi-user data isolation.
- Deployment on Streamlit Cloud.

12. Current Scale Assessment

The current project is at advanced prototype/BTP demo scale.

What is strong:

- Four-module architecture exists.
- Routing is centralized.
- Finance and tutor local models exist.
- Real market/equity data paths exist.
- Tests pass.
- The app can be run locally.
- The design avoids requiring a GPU.

What is not production scale yet:

- No authentication.
- No production database migration story.
- No robust observability.
- No compliance review layer.
- Data sources are partly scraped or unofficial.
- Synthetic data is used for core finance/tutor training.
- Avatar is not professional quality.
- Lip sync is not production-grade.
- Deployment model artifacts need explicit handling.

13. Build Roadmap by Topic

13.1 Finance

Build next:

- Real transaction ingestion.
- Better profile completeness flow.
- Financial goal planning.
- Portfolio/risk suitability questionnaire.
- Explainable risk dashboard.
- Model evaluation report.
- Strong guardrails for advice language.

13.2 Tutor

Build next:

- Full quiz-answer loop in UI.
- Larger concept graph.
- Revision scheduler.
- Learning analytics dashboard.
- Expert-reviewed explanations.
- More Indian finance examples.

13.3 Market Analysis

Build next:

- More reliable market data provider.
- Timestamped data display.
- Technical indicators.
- More sector/index coverage.
- News clustering and source attribution.
- Cache expiry and retry strategy.

13.4 Equity Research

Build next:

- Company master database.
- More ratios and trend calculations.
- Source-backed answers with fiscal-year tables.
- Robust parser for Screener changes.
- Peer comparison.
- Valuation summary without unsafe buy/sell calls.

13.5 Voice

Build next:

- Streaming TTS.
- Better voice settings.
- Hindi/Indian-English handling.
- Transcription confidence.
- Audio cleanup.

13.6 Avatar

Build next:

- Replace cartoon/anime GLB with professional avatar.
- Add correct blendshapes.
- Use phoneme/viseme timings.
- Improve lighting and camera.
- Add expression states.
- Add diagnostics and fallback UX.

13.7 UI/UX

Build next:

- Better professional visual system.
- Source and timestamp panels.
- Better mobile layout.
- Loading skeletons and error states.
- Separate developer/debug info from user-facing output.

13.8 Deployment

Build next:

- Model artifact deployment plan.
- Secrets management.

- Startup health checks.
- App logging.
- Cache management.
- CI test workflow.

14. Honest Conclusion

ARIA's core backend is meaningful and demonstrable: it has multiple working modules, local ML models, real market/equity data paths, speech support, and a passing automated test suite.

The weakest part is the avatar. It currently makes the project look less serious because the selected 3D model has a cartoon/anime feel and the lip sync is fragile. This should not be presented as a finished professional avatar. It should be presented as an experimental free browser avatar layer until it is replaced with a professional model and a proper viseme-driven animation pipeline.

Priority order from here:

1. Fix avatar quality or temporarily disable it for formal demos.
2. Strengthen source transparency and data timestamps.
3. Improve tutor quiz interaction and finance profile workflows.
4. Add deployment and model artifact automation.
5. Build production-level evaluation and compliance guardrails.